

Základní principy OOP jsou:

- Zapouzdření: ukrývání dat a implementace (za přístupovými metodami).
- Dědičnost: možnost tvořit nové třídy odvozením od již existujících tříd (nadřazených tříd) a tím logicky rozvíjet jejich vlastnosti.
 - + Abstrakce
 - V C-like jazycích + rozhraní
- Polymorfismus: možnost vytvářet objekty různých tříd, které se ale dokáží chovat jako stejné objekty (úzce spojeno s dědičností).

Obsah:

- Zapouzdření
 - a. Přístupové kvalifikátory
 - b. Property, getter, setter
- Dědičnost
 - a. Princip dědění
 - b. Metoda *super*
- Polymorfismus
 - a. Princip polymorfismu – „víceotypovost“
 - b. Překrývání metod
- Abstrakce
 - a. Abstraktní třída
 - b. Abstraktní metoda
- Statické vlastnosti
 - a. Statický atribut
 - b. Statická metoda

Zapouzdření: pomocí přístupových kvalifikátorů (slovně „public“, „protected“ a „private“) a getterů, setterů (@property, @atribut.setter).

- Přístupové kvalifikátory se v Pythonu odvíjí od předpony v názvu:
 - Public: žádná předpona (*self.atribut*), neomezený přístup odkudkoliv
 - Protected: podtržítka předponou (*self._atribut*), přístup pro sebe a potomky
 - Private: dvě podtržítka předponou (*self.__atribut*), přístup jen uvnitř dané třídy
- Stejný princip platí i pro metody.
 - Můžeme tedy vytvářet soukromé (nejdou volat mimo třídu) a chráněné metody.

Property (česky vlastnost) je druh metody, která „nemá parametry“ (kromě *self*). Na rozdíl od obyčejných metod se nevolá se závorkami (tudíž v ukázce jen *instance.kalorie*).

```
class Zvire:
    def __init__(self) -> None:
        self.__kalorie: int = 0

    @property
    def kalorie(self):
        return self.__kalorie

    @property
    def kalorie_text(self) -> str:
        return f"{self.__kalorie} kcal"

print(zvire.kalorie)
```

- Properties (vlastnosti třídy) se dají využít k vrácení nějaké zpracované hodnoty nebo ke zveřejnění **čtecího** přístupu k *private* atributu (viz výše – zápisové právo zůstává *private*).
- Property, která jen vrací hodnotu atributu, se zpravidla říká *getter*.

Setter slouží k nastavování hodnot (operátor „=" na atributu). Slouží k validaci nastavované hodnoty nebo k vyslání zprávy, že se atribut změnil.

- Vytvoření přes metodu se stejným názvem jako atribut a anotací „@<název>.setter“.

```
@kalorie.setter
def kalorie(self, hodnota: int):
    if hodnota > 0:
        self.__kalorie = hodnota
    else:
        self.__kalorie = 0
    print(„Kalorie se změnil“)
```

```
zvire.kalorie = -40
```

Dědičnost: pomocí závorek za názvem třídy při její definici, kdy třída odvozená od jiné třídy získává veškeré vlastnosti a metody této třídy. Na rozdíl od jiných jazyků, Python umožňuje dědit od více tříd (v Javě a podobných jen přes tzv. *rozhraní (interface)*).

- Znamená to, že dceřiná třída (též také „zděděná třída“ nebo „potomek“) zdědí od rodičovské třídy všechny její *public/protected* metody, vlastnosti a atributy. Navíc **musí** volat rodičovský konstruktor a vyplnit jeho parametry.
- Přístup na rodičovskou třídu (metody, atributy) přes zabudovanou metodu *super()*.

```
class Pes(Zvire):  
    def __init__(self, jmeno: str, plemeno: str):  
        # volání rodičovského konstruktora – povinné!  
        super().__init__(jmeno)  
  
        # navíc si může přidat své vlastní atributy (a metody)  
        self.plemeno = plemeno
```

- Vícenásobná dědičnost je v Pythonu možná, ale její komplexita je nad rámec letošního učiva. V C-like jazycích (do nichž patří Java) nelze dědit z více tříd, tudíž to pro zbytek studia ani nemá valný smysl. V praxi se navíc tento koncept příliš nedoporučuje (když už tak v upravené striktnější formě).

Polymorfismus: umožňuje se třídě **typově** vydávat za všechny její rodiče. Znamená to, že všechny třídy jsou *object*, samy sebou a případně rodiči, od kterých dědí.

- V příkladu výše je možné využít seznam s rodičovským typem:
 - Třída *Pes* dědí z třídy *Zvire*, tudíž se instance třídy *Pes* mohou používat i jako typ *Zvire* (a třídy *object*).
 - Neplatí to ale naopak! Třída *Zvire* se nemůže vydávat za třídu *Pes*, protože nemusí (a pravděpodobně nemá) všechny její vlastnosti.

```
pes1: Zvire = Zvire()  
pes2: Pes = Pes("Voříšek")  
list[Zvire] = [pes1, pes2]
```

- V C-like jazycích se na „obecný seznam“ (seznam, který může obsahovat cokoliv) používá seznam instancí třídy *Object* (např. `ArrayList<Object>`). To stejné jde i v jazyce Python, ale lepší je použít datový typ *Any* (např. `list[Any]`, popř. `list[object]`).

Další a zdaleka důležitější vlastností je tzv. **překrývání metod**, které polymorfismus umožňuje.

- Ve zděděné třídě můžeme (nesoukromé) metody překrývat. Pokud v ní nadefinujeme novou metodu, která má stejný prototyp (stejný název, parametry a návratový typ) jako ta v rodičovské třídě.
- Při volání z rodičovského i vlastního typu se vždy zavolá překrytá metoda.

```
class ZakladniSeznam:
    def __init__(self) -> None:
        self._seznam: list[Any] = []

    def pridat(self, prvek: Any) -> None:
        self._seznam.append(prvek)

    def delka(self) -> int:
        return len(self._seznam)

class RychlejsiSeznam(ZakladniSeznam):
    def __init__(self) -> None:
        super().__init__()
        self.__pocet: int = 0

    def pridat(self, prvek: Any) -> None:
        self._seznam.append(prvek)
        self.__pocet += 1

    # stejný prototyp funkce (stejný název, parametry a návratový typ)
    def delka(self) -> int:
        return self.__pocet
```

Abstraktní třídy jsou třídy, které nelze instancovat, ale slouží jako šablona pro odvozené třídy. Tyto třídy mohou mít neimplementované metody, které pak musí být implementovány v odvozených třídách. Jak třída, tak neimplementované metody musí být označeny jako abstraktní.

- V Pythonu je nutné importovat základ abstraktních tříd (*ABC*) a anotaci abstraktních metod (*@abstractmethod*). Obojí je obsaženo v modulu *abc*.

```
from abc import ABC, abstractmethod
```

```
class Tvar(ABC):  
    @abstractmethod  
    def obsah(self) -> float:  
        pass
```

- Takto definovaná třída nemůže být instancována a její abstraktní metody nemají tělo. Znamená to, že když se z této třídy dědí, tak se jejím abstraktním metodám musí doplnit tělo (musí všechny její abstraktní metody obsahovat, jinak se program nespustí).
- Vhodné využití je pro třídy, které mají sloužit jen jako šablona pro další třídy. Např. *Tvar* pro geometrické tvary (kruh, obdélník) nebo *Zvíře* pro druhy zvířat (pes, kočka). Nebo praktičtější využití pro kolekce – tudíž abstraktní třída „seznam“, ze které vytvoříme spojový seznam, cyklický seznam nebo rozšiřitelné pole.

Posledním principem využitelným ve třídách jsou statické atributy a metody:

Statické atributy patří přímo třídě (nepatří instancím). Znamená to, že každá instance této třídy vidí pouze na jednu sdílenou hodnotu daného atributu. Když jedna instance hodnotu změní, stejnou změnu uvidí i všechny ostatní instance.

- Jmennou konvencí se jejich názvy píšou celé velkými písmeny.
- Mohou také mít přístupové kvalifikátory (*protected*, *private*).
- Přístup k atributu je vždy přes název třídy.

Statické metody také patří třídám. Platí pro ně všechno stejné jako pro statické atributy.

- Musí se k nim uvést anotace `@staticmethod`. Anotace se nemusí importovat, je zabudovaná.

```
class KonecnySeznam:
    KAPACITA: int = 10
    __VKLADANE_CISLO: int = 0

    def __init__(self) -> None:
        self.__prvky: list[int] = []

    def pridat_prvek(self) -> None:
        if len(self.__prvky) < KonecnySeznam.KAPACITA:
            self.__prvky.append(KonecnySeznam.__VKLADANE_CISLO)
        else:
            print("Seznam je plný, nelze přidat prvek")

    @staticmethod
    def zmenit_vkladane_cislo(nove: int) -> None:
        KonecnySeznam.__VKLADANE_CISLO = nove

    def vypsati(self) -> None:
        print(self.__prvky, ":", len(self.__prvky))

s = KonecnySeznam()
s.pridat_prvek()
KonecnySeznam.zmenit_vkladane_cislo(5)
s.vypsati()
for i in range(8):
    s.pridat_prvek()
s.vypsati()

print(KonecnySeznam.KAPACITA)
```